

## Prérequis:

- **Java 25** ou supérieur
- **Javafx 25** ou supérieur spécifique à votre OS dans un dossier lib/

## Lancer le projet:

```
java -jar archidu7.jar
```

## Rappel sur le projet:

Le but de ce projet est de modéliser et simuler des circuits logiques à partir de leur description. La description des circuits se fait grâce au langage shdl qui nous a été présenté au début de l'année.

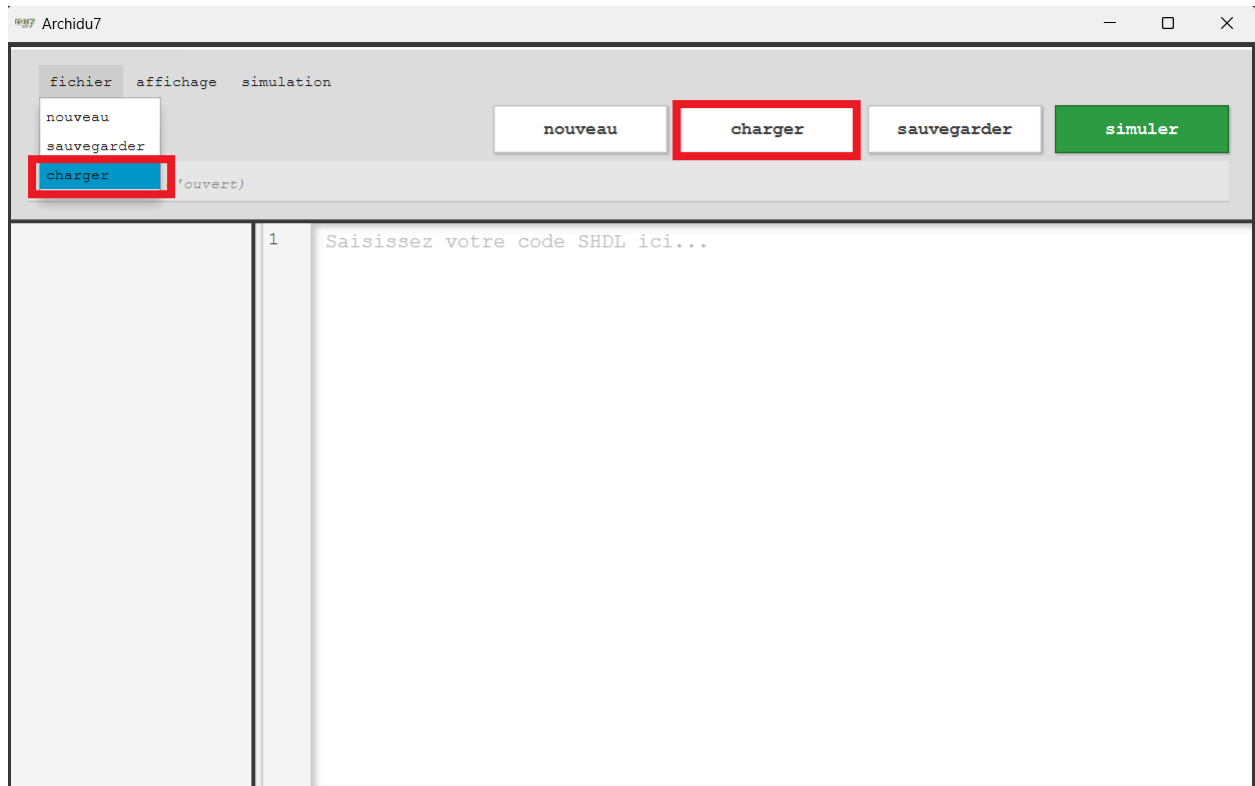
Pour utiliser ce service notre projet propose de séparer différentes simulation en “ modules “ , très similaire à ce est proposé sur les plateformes que l'on a utilisé en premiers semestre. Ce découpage en module permet l'appel à un module déjà écrit, ce qui permet de fragmenter et répartir la complexité d'un projet.

Pour ce qui est du langage en lui-même, nous vous invitons à consulter la doc disponible sur [shdl.fr](http://shdl.fr).

Pour ce qui est de l'utilisation de notre logiciel, nous vous invitons à lire ce qui suit.

## Charger son dossier de modules SHDL:

Par défaut, l'application cherche ses modules SHDL dans un dossier modules/ du répertoire courant. S'il n'existe pas, il est créé.  
Pour changer de répertoire, il faut utiliser charger situé dans la barre de bouton ou dans le menu fichier.

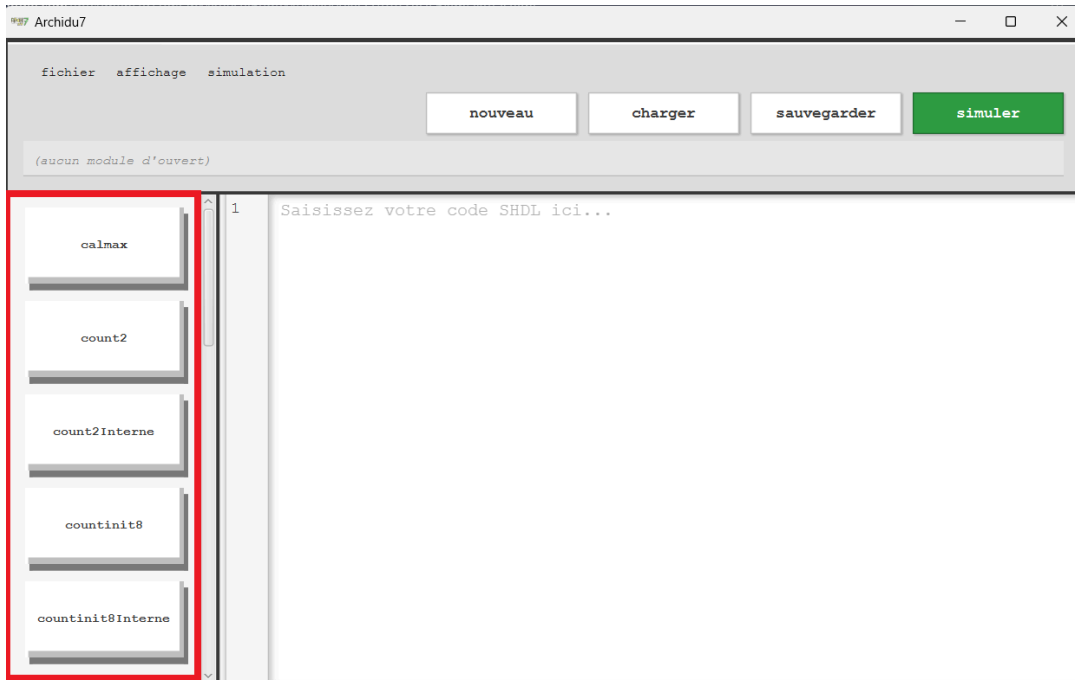


Sélectionner un dossier qui contient un dossier modules/ avec tous les fichier shdl ou le dossier modules directement.

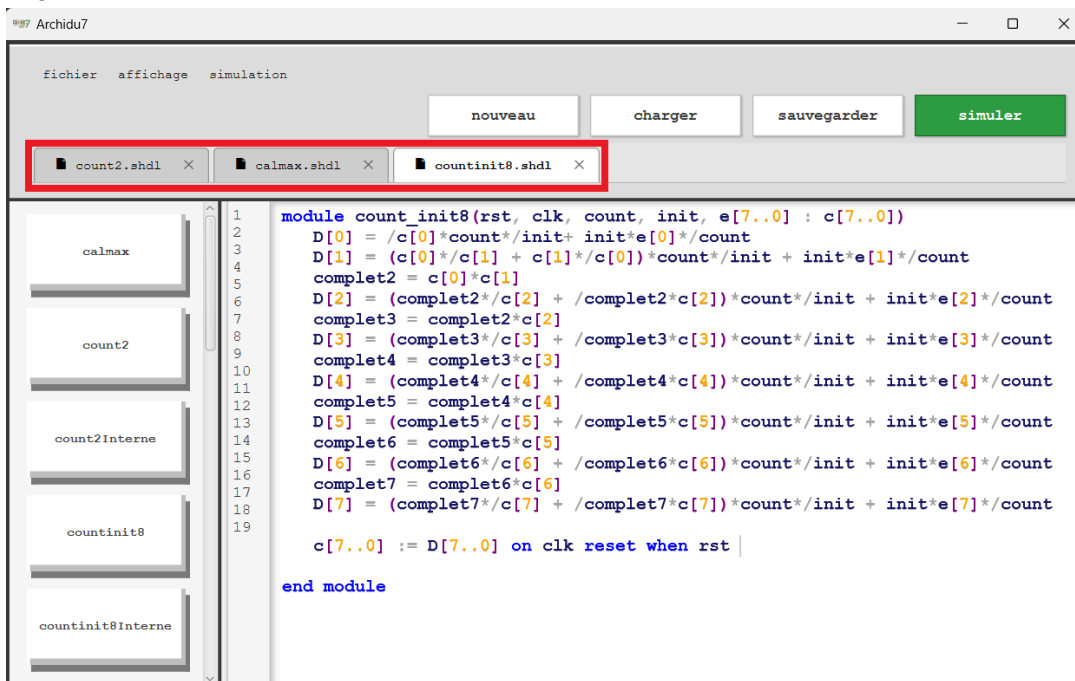
Seuls les fichiers shdl du dossier modules/ apparaîtront.

## Sélectionner un module à éditer:

Sélectionner le module que vous voulez éditer dans la liste à gauche.



Vous pouvez naviguer plus facilement dans les modules précédemment ouverts avec les onglets qui s'ouvrent en haut.

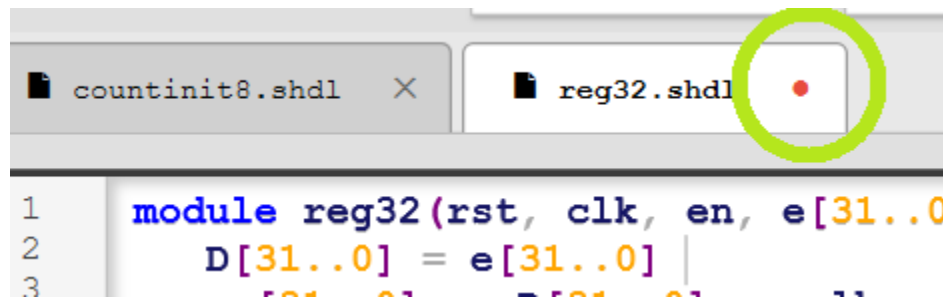


## Sauvegarder:

Vous pouvez sauvegarder à l'aide du bouton sauvegarder ou dans le menu fichier.

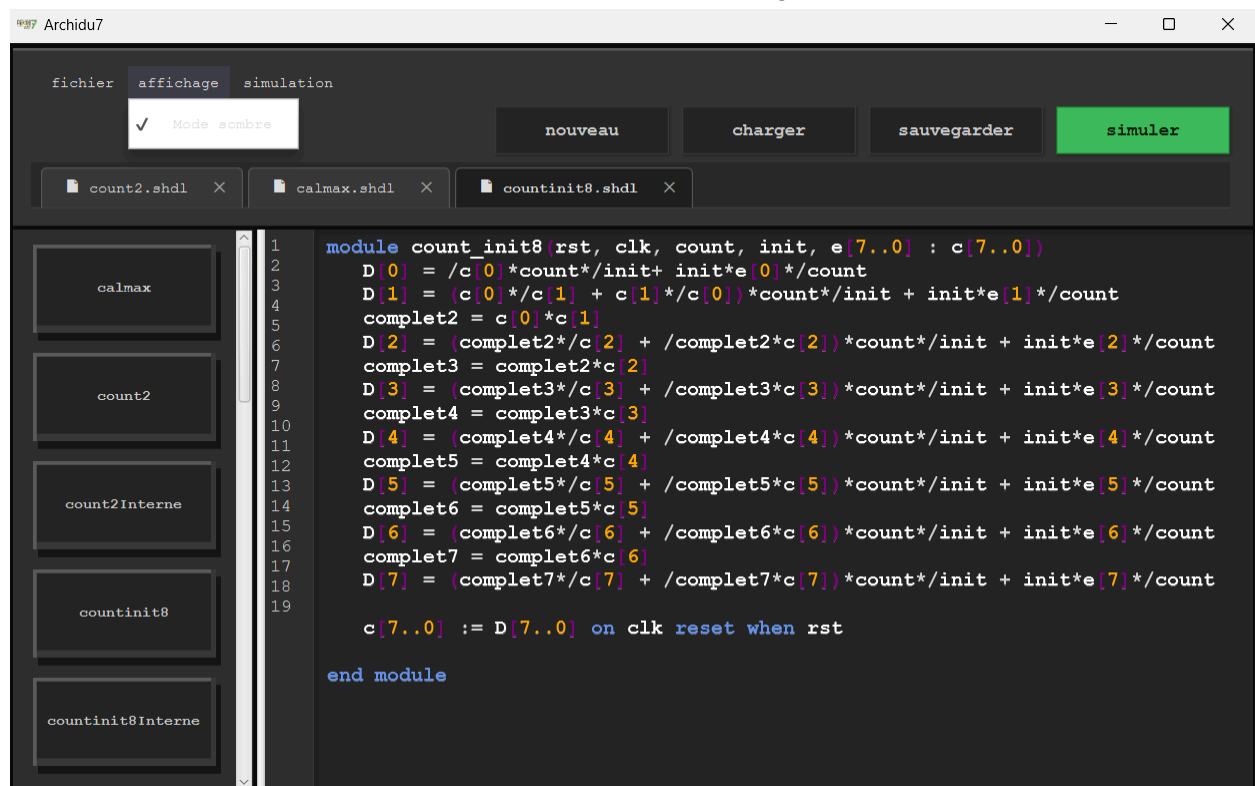
Si c'est un nouveau module, une fenêtre s'ouvrira pour demander le nom du module, **il est fortement recommandé de lui donner le même qu'à l'intérieur du script.**

Un point rouge indique que le module à subi des changements et doit être sauvegardé.



## Mode sombre:

Vous pouvez passer en mode sombre avec le menu affichage.



## Lancer la simulation:

Appuyer sur “simuler”, si votre code script est correcte une fenêtre s’ouvre, sinon l’erreur va s’afficher.

Erreur syntaxique a l'offset 857 : token inattendu (attendu ModuleKW) (trouve

```
D[0] = /c[0]*count*/init+ init*e[0]*/count
D[1] = (c[0]*/c[1] + c[1]*/c[0])*count*/init + init*e[1]*/count
complet2 = c[0]*c[1]
D[2] = (complet2*/c[2] + /complet2*c[2])*count*/init + init*e[2]*/count
complet3 = complet2*c[2]
D[3] = (complet3*/c[3] + /complet3*c[3])*count*/init + init*e[3]*/count
complet4 = complet3*c[3]
D[4] = (complet4*/c[4] + /complet4*c[4])*count*/init + init*e[4]*/count
complet5 = complet4*c[4]
D[5] = (complet5*/c[5] + /complet5*c[5])*count*/init + init*e[5]*/count
complet6 = complet5*c[5]
D[6] = (complet6*/c[6] + /complet6*c[6])*count*/init + init*e[6]*/count
complet7 = complet6*c[6]
D[7] = (complet7*/c[7] + /complet7*c[7])*count*/init + init*e[7]*/count

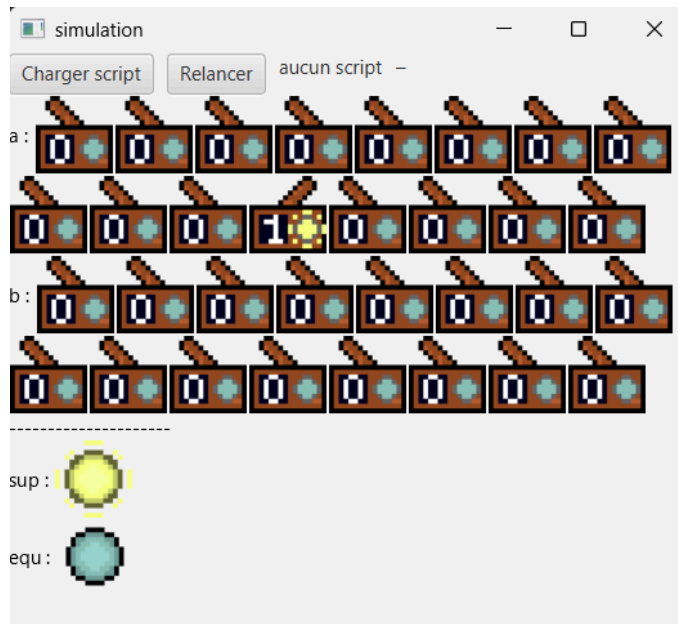
c[7..0] := D[7..0] on clk reset when rst

end modules
```

*Affichage d'une erreur*

## Simulation:

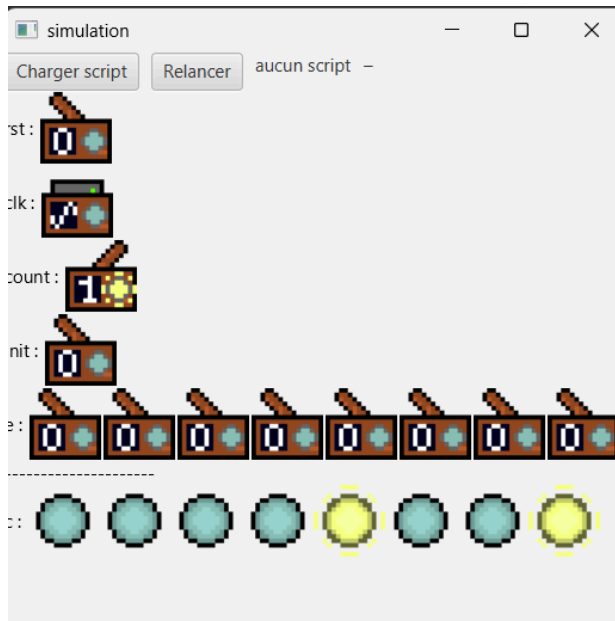
La simulation vous affiche des entrées et des sorties, cliquez sur une entrée pour changer son état.



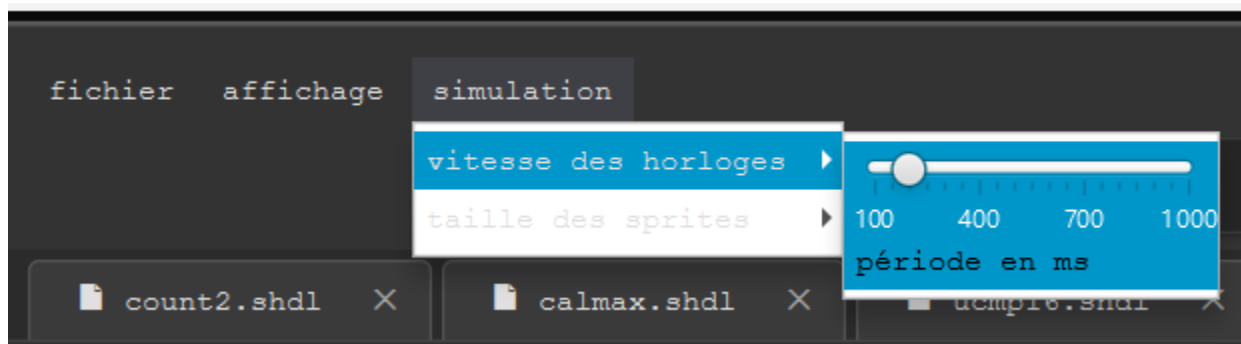
Vous pouvez changer la taille des sprite dans le menu simulation (il faut recharger la simulation pour que le changement s'applique)



Avec un clique droit, vous pouvez définir une entrée comme étant une horloge qui change d'état automatiquement périodiquement.



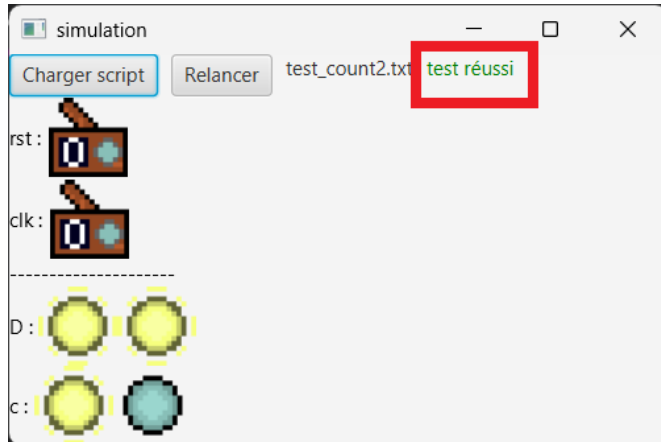
La période de l'horloge est paramétrable dans le menu simulation (il faut recharger la simulation pour être effectif)



## Scripts de tests:

Vous pouvez lancer un test avec le bouton charger script, puis sélectionner un test sous format .txt.

Relancer permet de refaire le test.



Pour effectuer un test, dans un script de test, une certaine syntaxe est nécessaire. Elle s'appuie sur le fait qu'un test se base sur une valeur affectée sur des entrées d'un module et des vérifications de valeur sur des sorties du module.

Le mot clé pour affecter une valeur est : set; tandis que pour vérifier le mot clé est : check.

La logique des test est séquentielle : tout se passe comme si les entrées étaient manipulées à la main. Les chefs se font au moment où ils sont appelés dans le script.

Les commentaires démarrent par //

Voici un exemple:

```
//init
set rst 1
set rst 0
//test
set clk 1
set clk 0
set clk 1
set clk 0
check c 10
```